

Politecnico di Milano

Anno accademico 2012-2013

Ingegneria del Software – Appello del 17 Settembre 2013

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Un'azienda di *catering* vuole proporre ricevimenti personalizzati per clienti onnivori, vegetariani e vegani. I clienti onnivori mangiano tutto, i vegetariani non mangiano carne e i vegani non mangiano né carne né derivati animali. L'azienda gestisce anche larchivio dei cibi che offre, organizzati secondo la stessa classificazione adottata per i clienti. Si noti che un cibo pu essere un prodotto di base, ad esempio frutta o verdura, o un piatto composto da diversi ingredienti (cibi) di base. Alcuni ingredienti posso essere mangiati anche da soli, mentre altri devono essere combinati per ottenere il prodotto finale.

Volendo organizzare i propri clienti per essere sicuri che ognuno mangi solo i cibi adatti, quante e quali classi definireste per cliente e cibo? Come mettereste in relazione clienti e cibi? Ovviamente, l'azienda non vuole gestire archivi separati per le diverse tipologie di cibi e clienti, ma vuole un solo archivio per i clienti e un solo archivio per i cibi, essendo però sicura che i vincoli imposti dalle diverse tipologie di clienti siano rispettati per costruzione.

a) Si utilizzi un diagramma delle classi UML per spiegare la soluzione adattata.

b) Supponendo che esistano una classe `Cliente` e una classe `Catering` (che include l'archivio dei clienti), si implementi un metodo `clientiVegani()` che restituisce un `Iterator<Cliente>` che consenta di scandire linsieme dei soli clienti vegani. A tale scopo, si richiede di definire sia la parte del rep della classe `Catering` che contiene l'archivio sia la classe che implementa `Iterator<Cliente>`.

c) Supponendo che, ad un certo punto, l'azienda voglia adottare un sistema di notifiche *push* verso i propri clienti per informarli delle nuove pietanze offerte (ovviamente sempre rispettando le diverse tipologie di clienti), come modifichereste il diagramma delle classi definito in precedenza? Quali metodi aggiungereste o cambiereste?

Esercizio 2

Si consideri il seguente programma Java

```
public class Figura {
   public void stampa(Figura f) {
      System.out.println("Figura");
   }
}

public class Quadrato extends Figura {
   public void stampa(Quadrato q) {
      System.out.println("Quadrato");
   }
}

public class Rettangolo extends Quadrato {
   public void stampa(Rettangolo r) {
      System.out.println("Rettangolo");
   }
   public void stampa(Quadrato r) {
      System.out.println("Rettangolo particolare");
   }
   public void stampa(Figura f) {
      System.out.println("Figura particolare");
   }
}

public class Test {
   public static void main(String args[]) {
      Figura f1, f2;
      Quadrato q = new Rettangolo();
      Rettangolo r = new Rettangolo();
      f1 = new Quadrato();
      f2 = new Rettangolo();

      f1.stampa(f2); ......................

      q.stampa(r);  ......................

      f1.stampa(q); ......................

      q.stampa(f1); ......................

      q.stampa(q); ......................
   }
}
```

Scrivere a lato delle precedenti istruzioni di stampa il risultato prodotto durante l'esecuzione. Cosa cambierebbe se `Rettangolo` estendesse `Figura` e non `Quadrato`? Motivare brevemente le risposte

Esercizio 3

Si consideri il seguente metodo generico `intersezione`, che prende come argomento due `ArrayList<T>` x e y, e un `Comparator` c. Gli ArrayList x e y contengono elementi di tipo generico T, che non sono modificati dal metodo, ma nessun valore null. Il metodo restituisce un nuovo `ArrayList<T>`, che contiene tutti e soli gli elementi che sono presenti sia in x che y, disposti in ordine crescente nell'ArrayList secondo l'ordinamento previsto dal Comparator.

```
public static <T> ArrayList<T> intersezione(ArrayList<T> x,
                                ArrayList<T> y, Comparator<? super T> c)
```

a) Si specifichi in JML il metodo intersezione. Si ricorda che l'interfaccia Comparator ha la seguente dichiarazione:

```
public interface Comparator<T> {
 int compare (Object obj1, Object obj2) throws  ClassCastException;
//\signals ClassCastException se obj1 e obj2 non sono di tipi confrontabili
//con il Comparator corrente.
//\result e' -1 se obj1<obj2, 0 se obj1.equals(obj2), +1 altrimenti
}
```

Esercizio 4

Si consideri il seguente frammento di codice Java, in cui le varie istruzioni sono state numerate per comodità:

```
public static int test(int x, int y) {
  1. int z = 5;
  2.  do  {
  3.    if (z< x && y !=1)
  4.                  x = z/(y-1);
  5.    else return z;
  6.    z--;
  7. }
  8. while (z>2);
  9. return z;
}
```

1. Si definisca il diagramma del flusso di controllo.
2. Si definisca l'insieme minimo di casi di test per coprire:
   a) le istruzioni;
   b) le condizioni
   c) i cammini.